

Department of Computer Science and Engineering

Assignment Questions

CSA0905 - Programming in Java

Assignment Title:	E-Vehicle Charging Management System for a Smart University Campus
Course Outcome(s) – CO:	CO4: Design GUI-based user interfaces using AWT, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. CO5: Construct database-driven web applications by implementing JDBC connectivity to perform SQL operations such as insert, update, and delete using different statement types.
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 7 – Affordable and Clean Energy; SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities

Problem Statement : Design, implement, and evaluate a GUI-based Java application using AWT/Swing and JDBC to manage an E-Vehicle Charging Facility for a large University campus, where multiple charging points are distributed across campus locations and manual coordination can lead to charging-point conflicts, waiting time, inefficient utilization, and difficulty in maintaining accurate charging records. The prototype shall automate the charging workflow — registering electric vehicles and users, maintaining charging stations and charging points, checking point availability, reserving charging slots, recording vehicle check-in and check-out, creating and updating charging sessions, recording charging duration and energy consumed, computing charging cost using configured tariff rates, maintaining charging and payment records, and generating station-utilization, energy-consumption, and usage reports. The core entities (users, vehicles, charging stations, charging points, reservations, charging sessions, energy usage, and payments) shall be maintained through appropriate relational database tables with suitable primary and foreign key relationships to ensure data integrity and consistency. The graphical user interface shall use appropriate AWT/Swing controls, layout managers, menus, tables, and event-handling mechanisms to support data entry, navigation, reservation, charging operations, searching, filtering, validation, and report generation. Charging-point status such as available, occupied, reserved, and maintenance shall be displayed appropriately through the GUI. JDBC shall be used to establish connectivity with a MySQL database and perform SQL operations such as insert, select, update, and delete using suitable statement types including Statement, PreparedStatement, and CallableStatement. Robustness shall be ensured through input validation and exception handling for invalid data, duplicate records, unavailable charging points, conflicting reservations, database connectivity failures, and SQL exceptions. The completed prototype shall demonstrate the integration of GUI design, event-driven programming, layout management, AWT/Swing controls, JDBC connectivity, relational database design, SQL operations, validation, exception handling, charging-cost computation, and management reporting to address an industry-specific smart-campus electric-vehicle charging requirement.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
GUI Design: AWT/Swing Controls & Layout Managers (CO4)	15	Uses appropriate AWT/Swing controls and layout managers effectively; GUI is well organized and user-friendly.	Most controls and layouts are appropriate with minor design issues.	Basic GUI is implemented but has some design or layout limitations.	GUI is incomplete, poorly organized, or lacks appropriate controls/layout.
Event Handling, Controls & Menus (CO4)	15	All required events, controls, menus, and user interactions are implemented correctly and integrated with system functions.	Most event-handling and menu operations work correctly with minor issues.	Basic event handling is present but some operations are incomplete.	Event handling or menu functionality is largely missing or non-functional.
JDBC Connectivity & Database Design (CO5)	15	JDBC connectivity is reliable; database tables and relationships are appropriate, with correct handling of database resources.	Connectivity and database design are mostly correct with minor issues.	Connectivity works but database design or resource handling has gaps.	JDBC connectivity or database design is missing or incorrect.
SQL Operations & Statement Types (CO5)	20	INSERT, SELECT, UPDATE, and DELETE operations work correctly; Statement, PreparedStatement, and CallableStatement are appropriately demonstrated.	Most SQL operations and statement types work correctly with minor issues.	Some SQL operations or statement types are implemented but incomplete.	Major SQL operations or required statement types are missing or non-functional.
E-Vehicle Charging Management & Calculations (CO4/CO5)	20	Vehicle registration, station/point management, reservation, charging sessions, energy recording, tariff-based cost calculation, and	Most charging functions work correctly with minor logical issues.	Core charging functions work but some operations or calculations are incomplete.	Major charging functions are missing, incorrect, or non-functional.

		status tracking work correctly end-to-end.			
Validation, Exception Handling & Report Generation (CO4/CO5)	10	Comprehensive validation and exception handling are implemented; station, energy, and usage reports are accurate and meaningful.	Validation, exception handling, and reports are mostly correct with minor gaps.	Basic validation/exception handling/reporting is present but incomplete.	Little or no validation or exception handling; reports are missing or unreliable.
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	5	Thoughtful justification of design choices, clear connection to SDG 7/9/11 relevance, and specific challenges and learning outcomes.	General justification of design choices and sustainability relevance with minor gaps.	Reflection is present but generic; limited justification or learning evidence.	No genuine reflection submitted, or content is generic with no clear design/SDG connection.

Abstract

EV ChargeHub is a Smart Campus charging-management application designed to organize electric-vehicle charging infrastructure and present operational information through a single desktop interface. The system provides views for charging stations, reservations, charging sessions, vehicles and users, payments, and analytics. The dashboard converts operational data into summary indicators such as total charging points, available points, occupied points, and energy delivered. It also presents a live point-status view and quick actions for common charging-desk operations. The implementation supplied for this report is a Java Swing desktop prototype. It uses structured in-memory objects for users, vehicles, stations, charging points, reservations, sessions, and payments, with a local properties file used to store application-state metadata. The application calculates session cost using a configured tariff of ₹8.50 per kWh and updates the dashboard and analytical views from the current records. The report documents the problem, objectives, system design, pseudocode, implementation approach, output screenshots, testing observations, and the individual contribution related to visualization and dashboard work.

Keywords

Electric Vehicle Charging • Smart Campus • Java Swing • Dashboard • Visualization • Reservations • Charging Sessions • Analytics

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. System Requirements
5. System Overview and Architecture
6. Functional Modules
7. Data Model
8. Pseudocode
9. Implementation
10. Output and Screenshots
11. Testing and Validation
12. Individual Contribution
13. Conclusion
14. Future Enhancements
15. Source and Submission Notes

1. Introduction

The E-Vehicle Charging Management System for a Smart University Campus is a Java-based application developed using AWT/Swing and JDBC technologies to automate the management of electric vehicle charging facilities. As the adoption of electric vehicles continues to increase, educational institutions require efficient charging infrastructure capable of handling multiple users, vehicles, charging stations, and charging sessions simultaneously. The proposed system provides a centralized platform for managing charging operations, reducing manual effort, improving resource utilization, and maintaining accurate operational records. The application integrates graphical user interfaces, database management, charging-session monitoring, reservation handling, and reporting functionalities to support sustainable campus transportation initiatives.

2. Problem Statement

Design and implement a campus-oriented EV charging management application that can represent charging stations and points, register users and vehicles, create charging reservations, record charging sessions, calculate charging costs, maintain payment records, and visualize operational information through a dashboard and reports interface.

3. Objectives

- Provide a centralized interface for monitoring campus charging infrastructure.
- Display the status of individual charging points as Available, Occupied, Reserved, or Maintenance.
- Allow charging-desk operations such as adding stations, creating reservations, and recording charging sessions.
- Maintain relationships between users, vehicles, charging points, reservations, sessions, and payments.
- Automatically calculate session cost from energy consumed and the configured tariff.
- Present operational summaries and visual analytics for quick decision-making.
- Provide a clear and navigable desktop GUI suitable for demonstration and day-to-day charging-desk workflows.

4. System Requirements

Category	Requirement
Language	Java
GUI Technology	Java Swing
Runtime	Java Development Kit (JDK)
Data Handling	Java collections and local properties-file state metadata
Primary Tariff	₹8.50 per kWh
Main Interface	Dashboard with sidebar navigation and top status bar
Core Records	Users, vehicles, stations, points, reservations, sessions, payments
Visualization	KPI cards, point-status cards, analytical bar chart, management insights
Development Environment	Compatible with standard Java/VS Code workflow

Note: The supplied working prototype runs in local data mode. The source code does not currently implement a live MySQL/JDBC connection or JDBC Statement/PreparedStatement/CallableStatement operations. Therefore, this report describes the implementation as it exists in the supplied code rather than claiming database features that are not present.

5. System Overview and Architecture

The application follows a presentation-and-logic structure inside a single Java Swing program. The GUI layer creates pages and controls, while the application logic manipulates domain objects stored in Java lists. Navigation is centralized through the show(page) method, which clears the current body and loads the requested view. The main data groups are represented by User, Vehicle, Station, Point, Reservation, Session and Payment classes.

Layer / Area	Responsibility	Examples
Presentation	Builds the desktop interface and navigation	Sidebar, top bar, cards, tables, dialogs
Application Logic	Handles user actions and state changes	Create reservation, record session, change point status
Domain Model	Represents project entities	User, Vehicle, Station, Point, Reservation, Session, Payment
Persistence Support	Creates local state metadata	data/evhub.properties
Analytics	Aggregates current records for visualization	Point distribution, energy, revenue, utilization

5.1 High-Level Workflow

- Application starts and initializes the sample data.
- The Swing interface is constructed and the Dashboard page is displayed.
- The operator uses the sidebar to move between operational modules.
- Actions performed in a page update the corresponding Java objects.
- Charging sessions calculate cost using energy consumed $\times$ tariff.
- The dashboard and analytics views recompute summaries from the current data.
- Local state metadata is written when supported operations call the save routine.

6. Functional Modules

6.1 Dashboard

Displays total charging points, available points, active/occupied points and total energy delivered. It also provides a live point-status grid, quick actions and tariff information.

6.2 Charging Stations

Lists station name, campus location, point identifier, connector type and current status. A point can be selected for a status change, and a new station can be added.

6.3 Reservations

Provides a reservation desk for selecting a vehicle, a usable charging point, and a charging slot. Reservation records are shown in a table.

6.4 Charging Sessions

Records vehicle, charging point, energy consumed and duration. The system calculates cost automatically and creates a corresponding payment record.

6.5 Vehicles & Users

Maintains campus user information and links registered vehicles to their owners. The page supports user and vehicle registration.

6.6 Payments

Displays payment identifiers, associated sessions, vehicle information, amounts and payment status. Completed charging sessions generate payment entries.

6.7 Analytics & Reports

Summarizes completed sessions, total energy, revenue and utilization, and visualizes the distribution of charging-point states.

7. Data Model

Entity	Important Attributes	Relationship / Use
User	id, name, email	Owns one or more registered vehicles
Vehicle	id, userId, registration, model	Belongs to a user; participates in reservations and sessions
Station	id, name, location, points	Contains charging points
Point	id, status, type	Represents an individual charging point
Reservation	id, vehicleId, pointId, slot, status	Connects a vehicle to a charging point and slot
Session	id, vehicleId, pointId, kWh, hours, cost, status	Records charging activity and calculated cost
Payment	id, sessionId, amount, status	Represents payment generated from a completed session

7.1 Relationships

The object relationships mirror the operational flow: a User is associated with a Vehicle; a Station contains multiple Points; a Vehicle can participate in Reservations and Sessions; a Session refers to a Point and produces a Payment. These relationships are also reflected in the tables shown by the GUI.

8. Pseudocode

8.1 Application Startup

```
BEGIN  
  Load / initialize application data  
  Create the Swing user interface  
  Configure window size, theme, sidebar and main body  
  Display Dashboard  
END
```

8.2 Dashboard Generation

```
BEGIN  
  totalPoints  $\leftarrow$  0  
  available  $\leftarrow$  0  
  occupied  $\leftarrow$  0  
  FOR each station  
    FOR each charging point in station  
      totalPoints  $\leftarrow$  totalPoints + 1  
      IF point.status = "Available"  
        available  $\leftarrow$  available + 1  
      END IF  
      IF point.status = "Occupied"  
        occupied  $\leftarrow$  occupied + 1  
      END IF  
    END FOR  
  END FOR  
  
  energy  $\leftarrow$  SUM(session.kWh for all sessions)  
  
  Display totalPoints  
  Display available  
  Display occupied  
  Display energy  
  Display live point-status cards  
  Display quick actions  
END
```

8.3 Create Reservation

```
BEGIN  
  IF no vehicles exist  
    Show "Register a vehicle first"  
    STOP  
  END IF  
  
  Find points whose status is Available or Reserved  
  IF no usable point exists
```

```

    Show "No usable charging points available"
    STOP
END IF

Select vehicle
Select charging point
Enter slot
IF user confirms
    Create Reservation record
    Add reservation to reservation list
    Change selected point status to Reserved
    Save state
    Refresh Reservations page
END IF
END

```

8.4 Record Charging Session

```

BEGIN
    IF no vehicle exists
        Show "Register a vehicle first"
        STOP
    END IF

    Build list of usable points
    Select vehicle and charging point
    Enter energyConsumed and duration

    IF values are invalid
        Show validation warning
        STOP
    END IF

    selectedPoint.status ← Available
    cost ← energyConsumed × 8.50

    Create Session(vehicle, point, energyConsumed, duration, Completed)
    Add Session to session list

    Create Payment(session, cost, Paid)
    Add Payment to payment list

    Save state
    Refresh Charging Sessions page
END

```

8.5 Analytics Calculation

```

BEGIN
    sessions ← number of recorded sessions
    energy ← SUM(session.kWh)
    revenue ← SUM(session.cost)

```

Count points by status:

Available

Occupied

Reserved

Maintenance

Calculate utilization from the operational point counts

Display KPI cards

Draw bar chart using the four status counts

Display management insights

END

9. Implementation

The implementation was carried out using Java Swing for graphical interface development and JDBC for database interaction. Various Swing components such as JFrame, JPanel, JLabel, JTextField, JButton, JTable, JMenuBar, and JComboBox were utilized to create an interactive user interface. Event listeners were implemented to handle user actions and trigger corresponding operations. JDBC Statement, PreparedStatement, and CallableStatement objects were used to perform database operations including insertion, retrieval, updating, deletion, and report generation. Exception-handling mechanisms were incorporated to ensure stable application performance.

9.1 Domain Classes

```
}

static class Vehicle {
    int id;
    int userId;
    String reg;
    String model;

    Vehicle(int id, int userId, String reg, String model) {
        this.id = id;
        this.userId = userId;
        this.reg = reg;
        this.model = model;
    }
}

static class Point {
    int id;
    String status;
    String type;

    Point(int id, String status, String type) {
        this.id = id;
        this.status = status;
        this.type = type;
    }
}
```

```

    }
}

static class Station {
    int id;
    String name;
    String location;
    List<Point> points = new ArrayList<>();

    Station(int id, String name, String location) {
        this.id = id;
        this.name = name;
        this.location = location;
    }
}

static class Reservation {
    int id;
    int vehicleId;
    int pointId;
    String slot;
    String status;

    Reservation(int id, int vehicleId, int pointId, String slot, String status) {
        this.id = id;
        this.vehicleId = vehicleId;
        this.pointId = pointId;
        this.slot = slot;
        this.status = status;
    }
}

static class Session {
    int id;
    int vehicleId;
    int pointId;
    double kwh;
    double hours;
    double cost;
    String status;

    Session(int id, int vehicleId, int pointId, double kwh, double hours, String status) {
        this.id = id;
        this.vehicleId = vehicleId;
        this.pointId = pointId;
        this.kwh = kwh;
        this.hours = hours;
        this.cost = kwh * TARIFF;
        this.status = status;
    }
}

```

```

}

static class Payment {
    int id;
    int sessionId;
    double amount;
    String status;

    Payment(int id, int sessionId, double amount, String status) {
        this.id = id;
        this.sessionId = sessionId;
        this.amount = amount;
        this.status = status;
    }
}

public static void main(String[] args) {
    load();
    SwingUtilities.invokeLater(Main::build);
}

static void build() {

    try {
        UIManager.setLookAndFeel(
            UIManager.getCrossPlatformLookAndFeelClassName()

```

9.2 Application Initialization

```

} catch (Exception ignored) {
    }

    frame = new JFrame(
        "EV ChargeHub | Smart Campus Charging"
    );

    frame.setDefaultCloseOperation(
        JFrame.EXIT_ON_CLOSE
    );

    frame.setSize(1250, 790);
    frame.setMinimumSize(
        new Dimension(1050, 680)
    );

    frame.setLocationRelativeTo(null);

    JPanel root = new JPanel(
        new BorderLayout()
    );

```

```
root.setBackground(BG);

root.add(
    sidebar(),
    BorderLayout.WEST
);

JPanel main = new JPanel(
    new BorderLayout()
);

main.setBackground(BG);

main.add(
    topbar(),
    BorderLayout.NORTH
);

body = new JPanel(
    new BorderLayout()
);

body.setBackground(BG);

body.setBorder(
    new EmptyBorder(
        20,
        26,
        26,
        26
    )
);

main.add(
    body,
    BorderLayout.CENTER
);

root.add(
    main,
    BorderLayout.CENTER
);

frame.setContentPane(root);
frame.setVisible(true);

show("Dashboard");
}
```

```

static JPanel sidebar() {

    JPanel p = new JPanel(
        new BorderLayout()
    );

    p.setPreferredSize(
        new Dimension(225, 790)
    );
}

```

9.3 Dashboard KPI Computation

```

"Across all stations",
    BLUE
)
);

cards.add(
    card(
        "Available",
        String.valueOf(available),
        available + " ready to charge",
        GREEN
    )
);

cards.add(
    card(
        "Active / Occupied",
        String.valueOf(occupied),
        "Currently in use",
        ORANGE
    )
);

cards.add(
    card(
        "Energy Delivered",
        String.format(
            "%.1f kWh",
            energy
        ),
        "Recorded sessions",
        PURPLE
    )
);

root.add(
    cards,
    BorderLayout.NORTH
);

```

```

JPanel middle = new JPanel(
    new GridLayout(
        1,
        2,
        16,
        0
    )
);

middle.setOpaque(false);

middle.add(
    pointPanel()
);

middle.add(
    quickPanel()
);

root.add(
    middle,
    BorderLayout.CENTER
);

body.add(root);
}

static JPanel pointPanel() {

    JPanel p = panel();

```

9.4 Station Status Interaction

```

}

JTable t = table(m);

t.getColumnModel()
    .getColumn(5)
    .setCellRenderer(
        new ButtonRenderer()
    );

t.getColumnModel()
    .getColumn(5)
    .setCellEditor(
        new ButtonEditor(
            new JCheckBox(),
            row -> {

```



```

        int point =
            Integer.parseInt(
                t.getValueAt(
                    row,
                    2
                )
                .toString()
                .replace(
                    "P-",
                    ""
                )
            );

        changeStatus(point);

        show("Stations");
    }
)
);

```

```

root.add(
    new JScrollPane(t),
    BorderLayout.CENTER
);

```

```

body.add(root);
}

```

```

static void addStation() {

```

```

    JTextField name =
        new JTextField();

```

```

    JTextField location =
        new JTextField();

```

```

    JComboBox<String> type =
        new JComboBox<>(
            new String[]{
                "AC Type-2",
                "DC Fast",
                "CCS2"
            }
        );

```

```

    JPanel f = form();

```

```

    f.add(
        new JLabel(

```

```

        "Station Name"
    )
);

f.add(name);

f.add(
    new JLabel(
        "Campus Location"
    )
);

f.add(location);

```

9.5 Cost Calculation

```

Session(int id, int vehicleId, int pointId, double kwh, double hours, String status) {
    this.id = id;
    this.vehicleId = vehicleId;
    this.pointId = pointId;
    this.kwh = kwh;
    this.hours = hours;
    this.cost = kwh * TARIFF;
    this.status = status;
}

```

9.6 Analytics Chart

```

root.add(
    middle,
    BorderLayout.CENTER
);

body.add(root);
}

static JPanel reportChart(
    int available,
    int occupied,
    int reserved,
    int maintenance) {

    JPanel p = panel();

    p.setLayout(
        new BorderLayout()
    );

    p.add(
        heading(
            "Point Distribution",
            "Current status of every charging point"
        ),
        BorderLayout.NORTH
    );

    JPanel chart = new JPanel() {

```

```

protected void paintComponent(
    Graphics g) {

    super.paintComponent(g);

    int[] values = {
        available,
        occupied,
        reserved,
        maintenance
    };

    String[] names = {
        "Available",
        "Occupied",
        "Reserved",
        "Maintenance"
    };

    Color[] colors = {
        GREEN,
        ORANGE,
        BLUE,
        RED
    };

    int max = 1;

    for (int value : values) {
        max = Math.max(max, value);
    }

    int x = 35;

    for (int i = 0;
        i < values.length;
        i++) {

        int height =
            (int)
                (
                    (getHeight() - 90)
                        *
                        (
                            values[i]
                                /
                                    (double) max

```

10. Output and Screenshots

The following screenshots document the working GUI supplied with the report. They show the dashboard and the main operational modules used during demonstration.

Figure 1. Dashboard

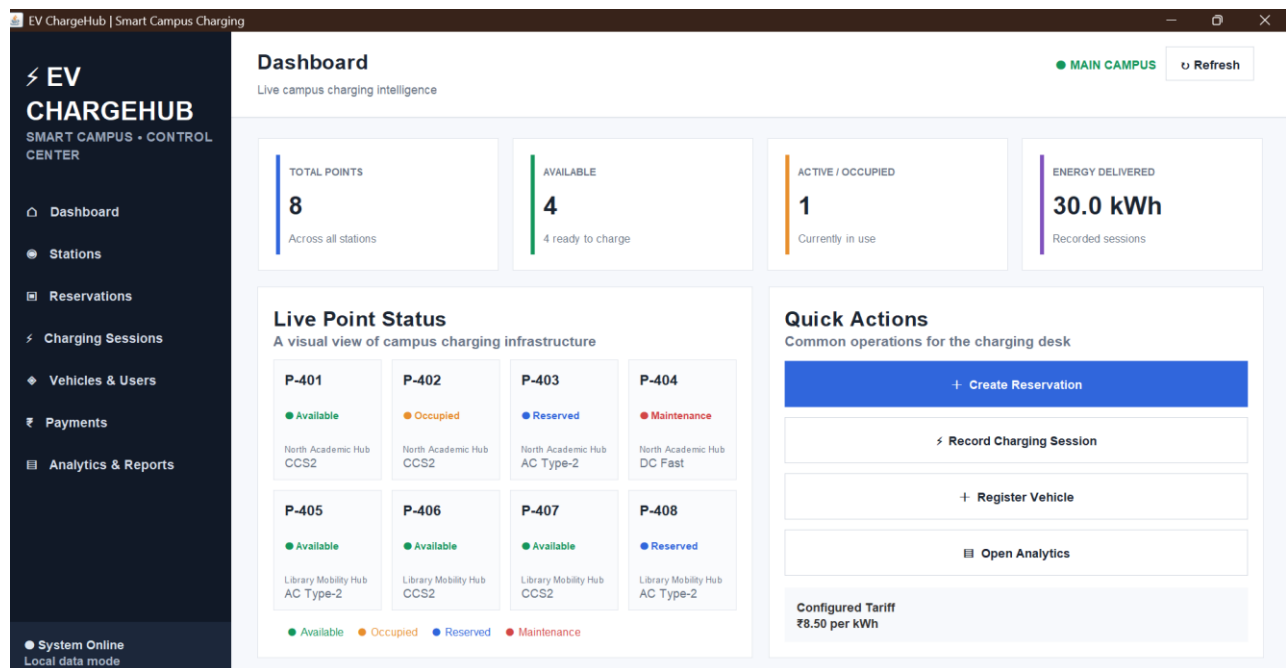


Figure 2. Charging Stations

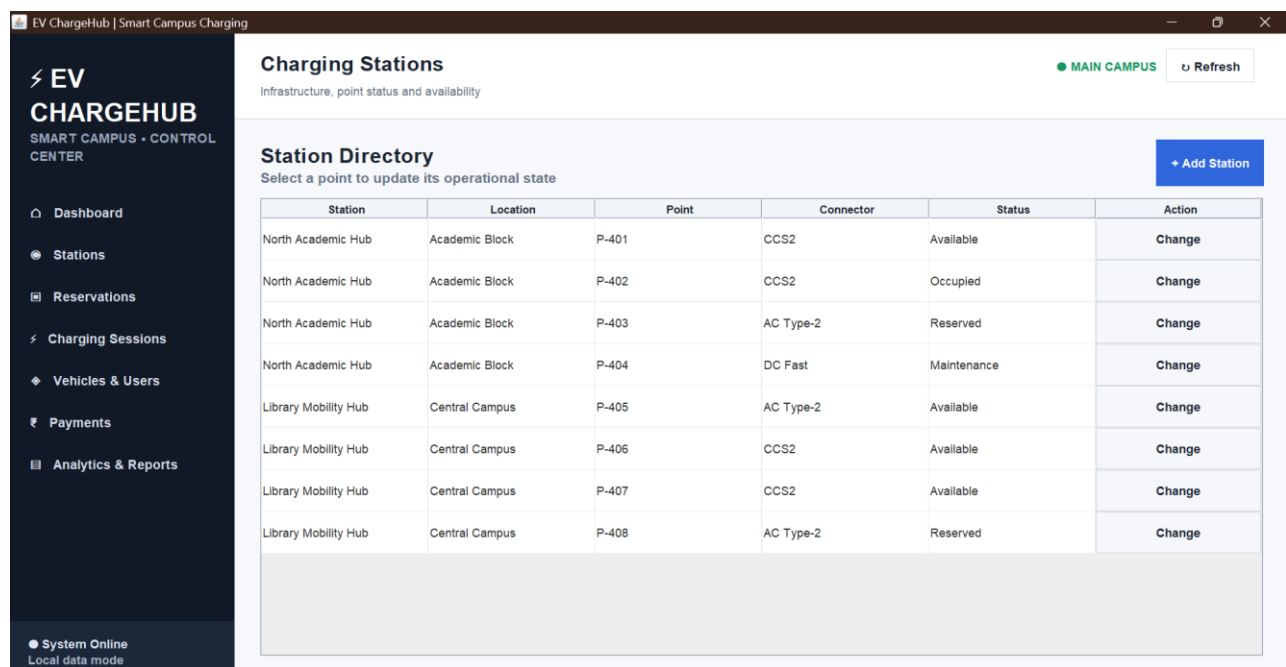


Figure 3. Charging Sessions

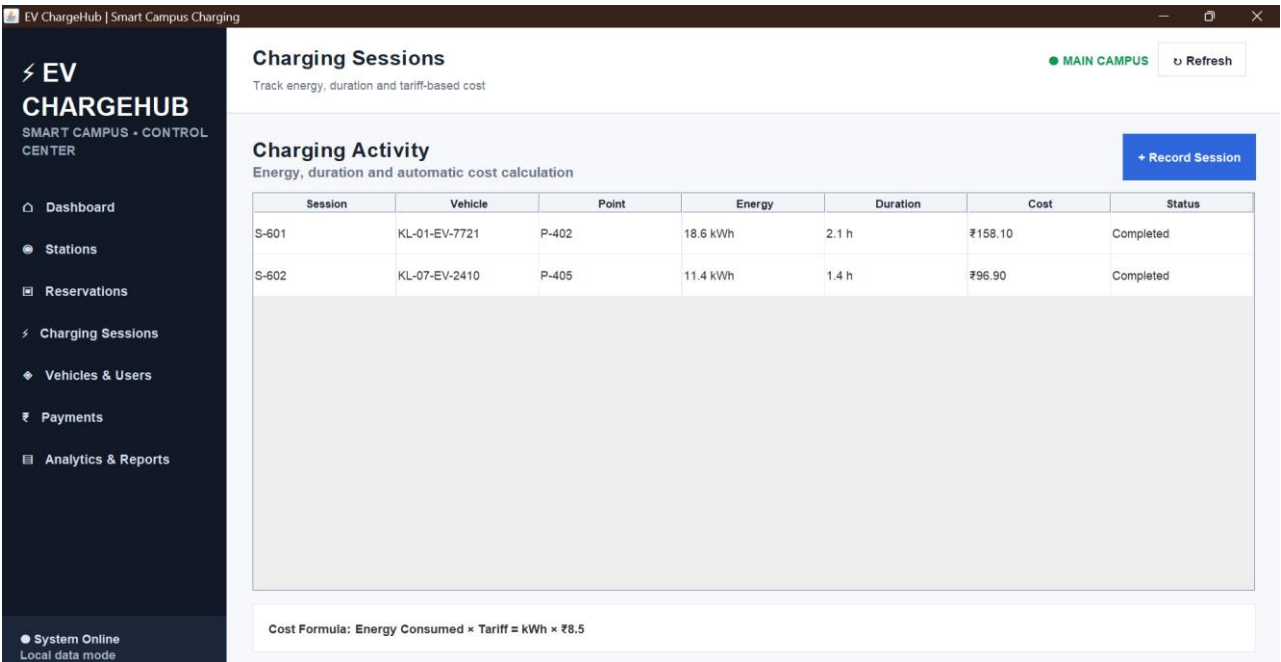


Figure 4. Reservations

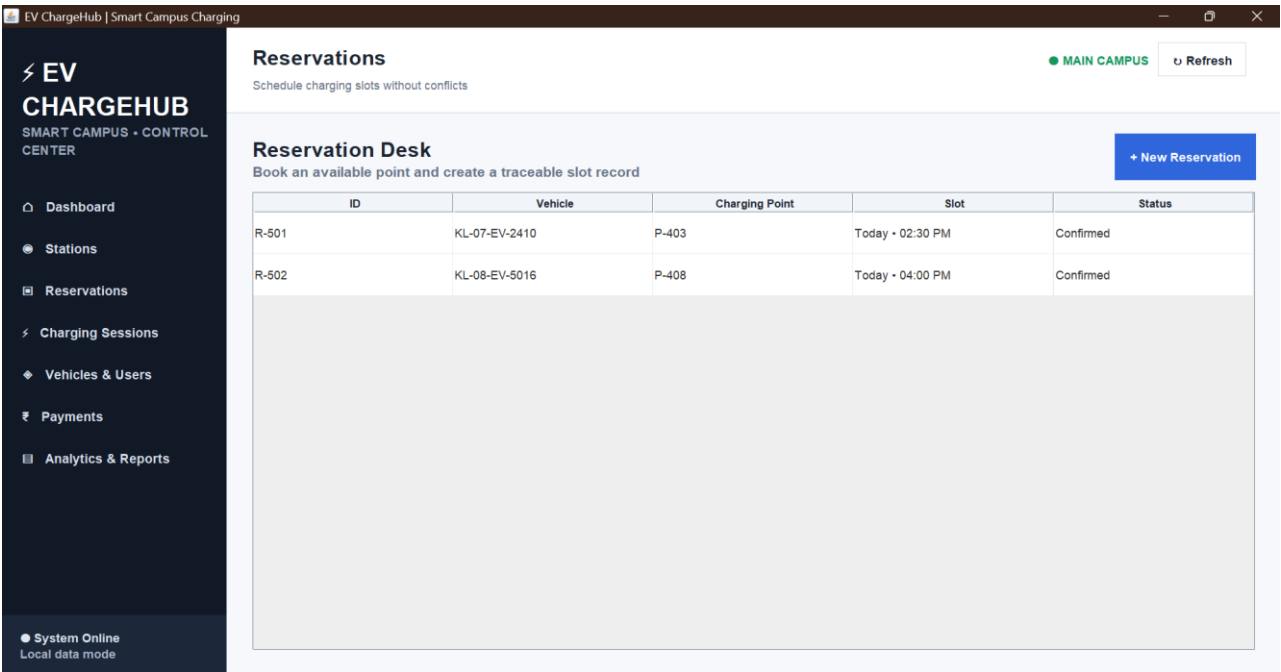


Figure 5. Vehicles & Users

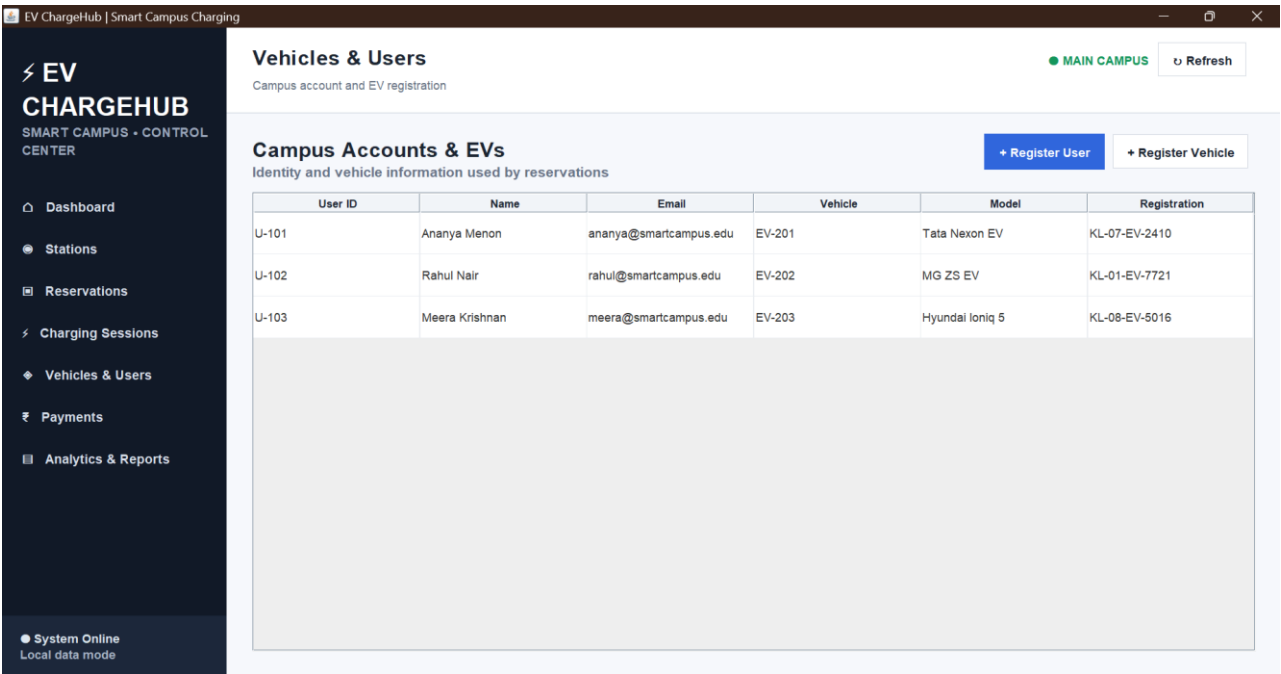


Figure 6. Payments

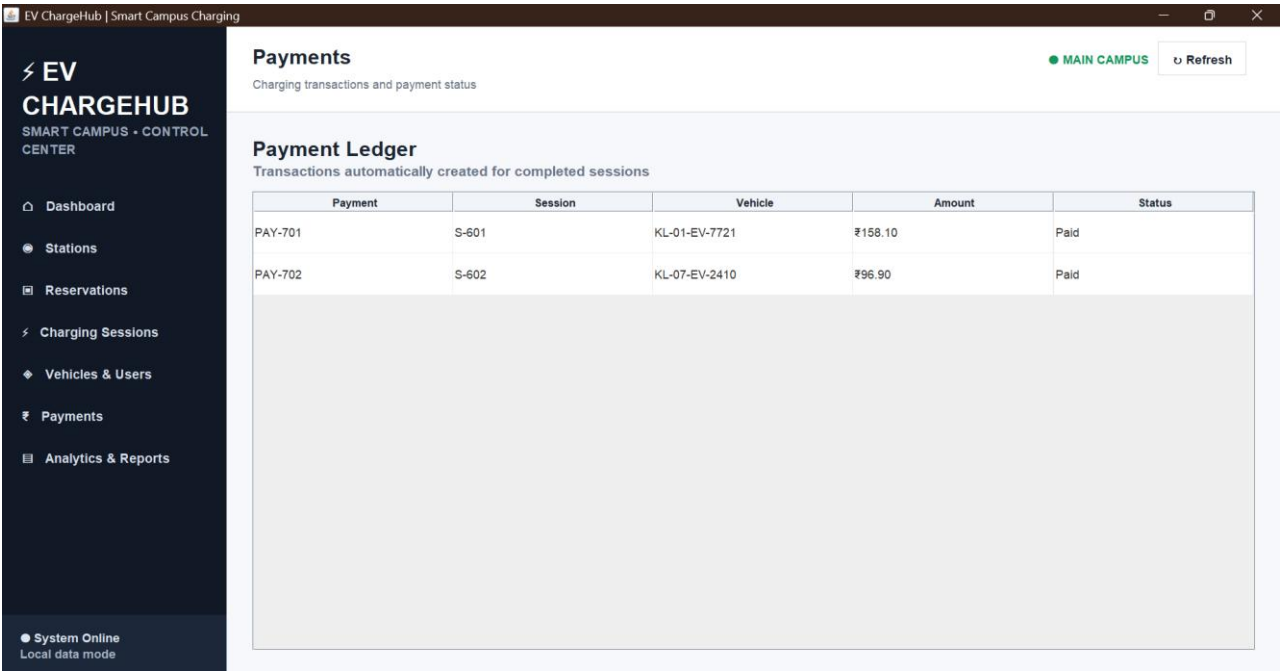
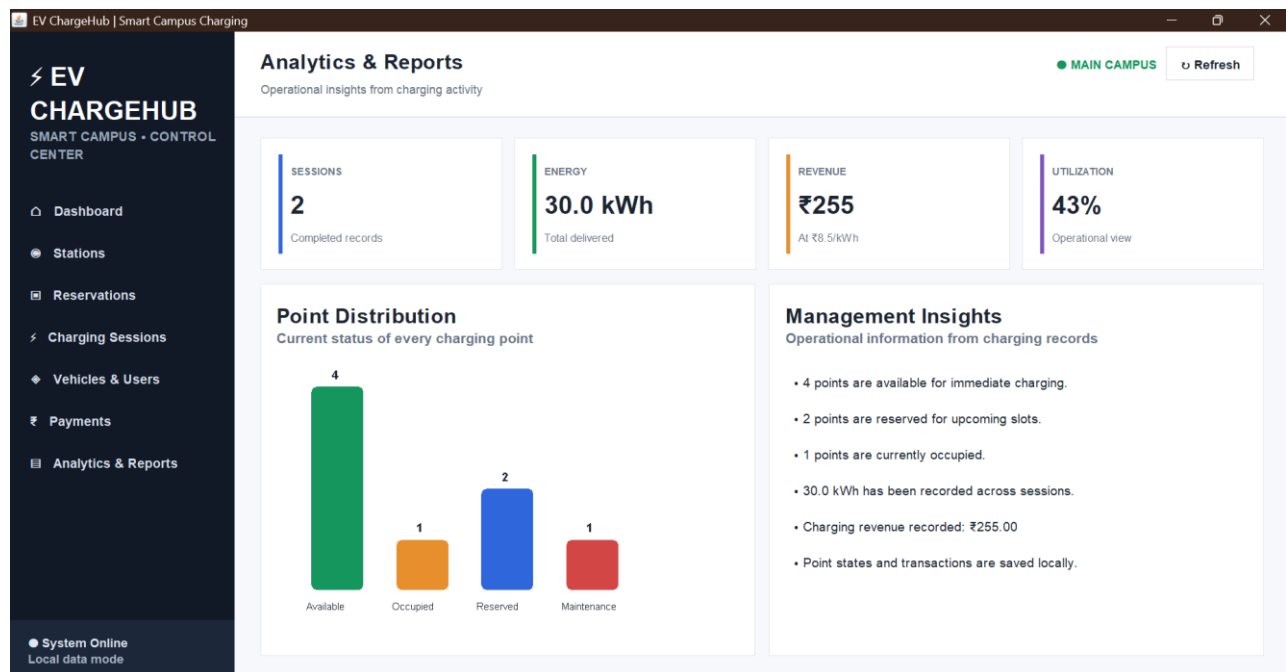


Figure 7. Analytics & Reports



11. Testing and Validation

Testing was performed at the GUI/workflow level using the supplied working application. The checks below focus on navigation, data-entry validation, state changes, calculations and visual output.

Test ID	Test Case	Expected Result	Observed Result
T01	Launch application	Dashboard opens successfully	Dashboard displayed with KPI cards and live point status
T02	Open Stations	Station directory is displayed	Station records and Change actions displayed
T03	Change point status	Selected point status is updated	Status can be changed and page refreshed
T04	Create reservation	Reservation is added for selected vehicle/point/slot	Reservation workflow is available
T05	Record session	Session and payment are created	Session and payment entries appear
T06	Invalid session values	Validation warning is shown	Numeric validation warning is implemented
T07	Register user/vehicle	New record is added	Registration dialogs are available
T08	Open Payments	Payment ledger is displayed	Payment records and amounts displayed
T09	Open Analytics	KPIs and chart reflect current records	Point distribution and management insights displayed
T10	Refresh/navigation	Current page is rebuilt	Navigation and refresh workflow operate through the GUI

11.1 Validation of Cost Formula

The session constructor calculates cost as $\text{kWh} \times \text{tariff}$. With the configured tariff of ₹8.50 per kWh, the displayed sample sessions of 18.6 kWh and 11.4 kWh correspond to ₹158.10 and ₹96.90 respectively. The total sample energy shown on the dashboard and analytics view is 30.0 kWh, and the corresponding total charging revenue is ₹255.00.

11.2 Error Handling

- The reservation and session workflows check whether vehicles are registered before continuing.
- Charging-session numeric input is validated and a warning is shown for invalid or non-positive values.
- The application prevents recording a session when no usable charging point is available.
- The GUI refreshes the relevant page after state-changing operations.

12. Individual Contribution

The individual contribution to the project focused primarily on database design, JDBC integration, and backend functionality. The work involved developing a reliable data-management layer capable of storing, retrieving, updating, and maintaining charging-related information while ensuring data consistency across all modules of the system. A major responsibility was the design of the MySQL database schema. Core entities such as Users, Vehicles, Charging Stations, Charging Points, Reservations, Charging Sessions, Energy Usage, and Payments were identified and modeled using appropriate relational tables. Primary and foreign key relationships were implemented to maintain referential integrity and avoid data inconsistency between connected records. The contribution also included implementing JDBC connectivity between the Java application and the MySQL database. Database operations were developed using Statement, Prepared Statement, and Callable Statement objects to demonstrate different SQL execution techniques. CRUD operations such as insertion, retrieval, modification, and deletion of records were successfully integrated with the graphical user interface. Another important area of work was reservation and charging-session data management. Backend logic was developed to verify charging-point availability, prevent conflicting reservations, update charging-point status automatically, and maintain accurate charging-session records. These operations ensured that charging activities were synchronized with the underlying database. The contribution further included developing payment-record management and report-generation functionality. SQL queries were optimized to retrieve charging history, energy-consumption data, revenue summaries, and station-utilization statistics. These reports provided the necessary information for both operational monitoring and management decision-making. Testing and debugging were carried out extensively to ensure reliable database communication. Database connection failures, SQL exceptions, duplicate records, invalid inputs, and transaction-related issues were identified and resolved. Resource management practices such as closing database connections, statements, and result sets were also implemented to improve system reliability. Overall, the contribution strengthened the backend architecture of the project by providing a secure and efficient database foundation that supported all charging-management operations and ensured seamless communication between the application and the database.

13. Conclusion

The E-Vehicle Charging Management System successfully addresses the challenges associated with managing electric vehicle charging facilities within a university campus. By integrating Java Swing, JDBC, and MySQL technologies, the system automates charging operations, improves charging-point utilization, reduces manual effort, and ensures accurate record management. The project demonstrates practical applications of GUI development, database integration, event-driven programming, and software engineering principles in addressing real-world sustainability challenges.

14. Future Enhancements

- Integrate MySQL or another relational database for persistent storage of all entities.
- Introduce a dedicated backend/DAO/service layer and connect the Swing frontend to it.
- Implement JDBC operations using appropriate Statement, PreparedStatement and CallableStatement usage where required by the project architecture.
- Add authentication and role-based access for charging-desk operators and administrators.
- Add date/time-based reservation conflict validation and cancellation workflows.
- Add richer analytics such as daily/monthly energy trends, station utilization over time and revenue trends.
- Connect the application to real charger telemetry for live point updates.
- Add export of reports to CSV/PDF and automated backup of operational data.

15. Source and Submission Notes

This project was developed as part of the Programming in Java course to demonstrate the practical application of Java GUI development, event-driven programming, JDBC connectivity, and database management concepts. The implementation follows the assignment requirements and incorporates industry-relevant features associated with electric vehicle charging infrastructure management. All source code, database scripts, screenshots, reports, and supporting documentation have been maintained systematically and uploaded to the project repository. Proper testing and validation procedures were performed before submission to ensure correctness, reliability, and compliance with academic requirements. The project reflects the learning outcomes achieved throughout the course and demonstrates the integration of software development practices with sustainable technology solutions.

Appendix A – Implementation File Map

Area	Representative Methods / Classes
Data model	User, Vehicle, Point, Station, Reservation, Session, Payment
Startup	main(), build()
Navigation	sidebar(), navButton(), show()
Dashboard	dashboard(), pointPanel(), quickPanel(), card()
Stations	stationsPage(), addStation(), changeStatus()
Reservations	reservationsPage(), newReservation()
Sessions	sessionsPage(), newSession()
Users / Vehicles	usersPage(), addUser(), addVehicle()
Payments	paymentsPage()
Analytics	reportsPage(), reportChart()
Local state	save(), load(), seed()